

UML 2.0 – GUIA COMPLETO

Diagrama de Componentes UML

Modelando a arquitetura de sistemas reais

CONCEITOS

SÍMBOLOS

EXEMPLOS PRÁTICOS

Do modelo à implementação – passo a passo

Por Que Precisamos Deste Diagrama?

Antes de escrever uma única linha de código, todo arquiteto de software precisa responder a perguntas fundamentais sobre a estrutura do sistema. Ignorá-las cedo gera dívida técnica cara de pagar depois.

Quais módulos existirão?

Identificar as unidades funcionais independentes do sistema antes de começar a implementação.

Como eles se comunicam?

Definir contratos e interfaces entre os módulos para garantir baixo acoplamento e alta coesão.

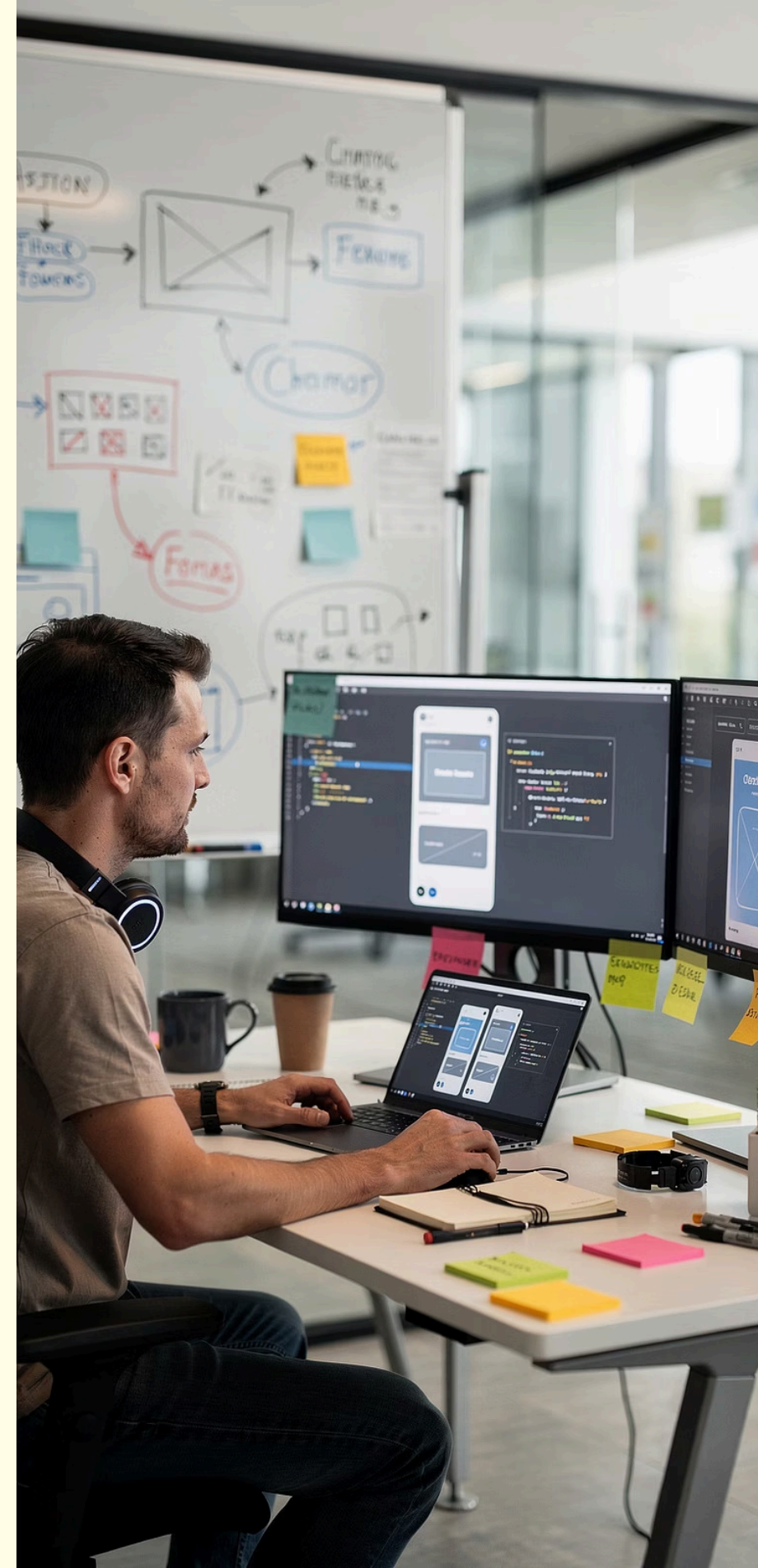
Quem depende de quem?

Mapear as dependências para evitar ciclos, facilitar testes e planejar a ordem de desenvolvimento.

O que pode ser reutilizado?

Encontrar componentes genéricos que podem ser compartilhados entre diferentes partes do sistema.

 O Diagrama de Componentes UML foi criado exatamente para responder essas perguntas de forma visual, padronizada e colaborativa.



O Que É um Componente?

Definição UML

Segundo a especificação UML, um **componente** é uma unidade modular e independente que encapsula seu conteúdo interno e se comunica com o mundo exterior exclusivamente por meio de interfaces bem definidas. Ele pode ser substituído por outro componente compatível sem impactar o restante do sistema.

A ideia central é **encapsulamento + substituíbilidade**.

Exemplos do Mundo Real

API de
Autenticação

Serviço de
Pagamento

Módulo CRM

Sistema
Acadêmico

Biblioteca de Relatórios

Cada um desses exemplos encapsula uma responsabilidade clara e expõe interfaces para que outros módulos possam consumi-lo.

O Que o Diagrama Mostra – e o Que Ele Não Mostra

Compreender o escopo do Diagrama de Componentes é essencial para utilizá-lo no momento certo do projeto.

✓ O que ele representa

- **Estrutura do sistema** – a organização arquitetural em alto nível
- **Módulos principais** – as unidades funcionais e seus nomes
- **Dependências** – quem usa quem, representadas por setas tracejadas
- **Interfaces fornecidas** – serviços que o componente oferece ao exterior
- **Interfaces requeridas** – serviços que o componente precisa consumir

✗ O que ele não representa

- **Fluxos de dados** – para isso, use Diagramas de Atividade ou Sequência
- **Sequência de execução** – ordem de chamadas vai para o Diagrama de Sequência
- **Algoritmos e lógica** – detalhes internos ficam no Diagrama de Classes



Misturar fluxo com arquitetura é um dos erros mais comuns. Use o diagrama certo para cada questão.

Onde o Diagrama de Componentes se Encaixa?

O Diagrama de Componentes ocupa uma posição estratégica no ciclo de modelagem UML – ele é a ponte entre a análise de negócio e a implementação técnica.



Note que os Componentes aparecem **depois** de Classes e **antes** da Arquitetura física – eles traduzem o modelo de domínio em módulos deployáveis. Este é o momento em que decisões estruturais têm maior impacto e menor custo de mudança.

Benefícios do Diagrama de Componentes

Segundo o Lucidchart e a comunidade de arquitetura de software, o Diagrama de Componentes oferece vantagens concretas em todas as fases do ciclo de vida do sistema.



Visibilidade Estrutural

Permite que toda a equipe enxergue a organização do sistema num único diagrama, facilitando onboarding e alinhamento técnico.



Entendimento de Relacionamentos

Torna explícito quem depende de quem, evitando surpresas durante integração e testes.



Identificação de Interfaces

Define contratos claros entre módulos, promovendo baixo acoplamento e maior testabilidade.



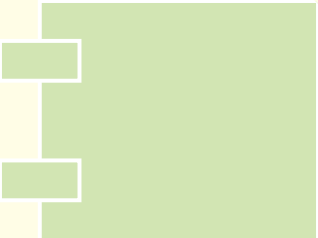
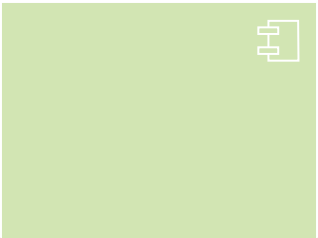
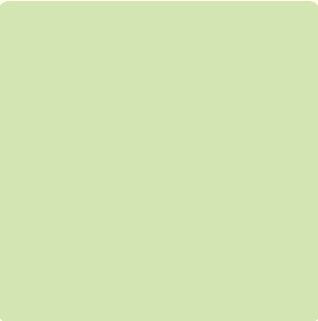
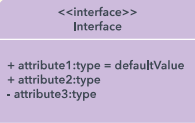
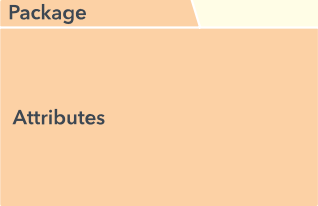
Facilita Manutenção e Evolução

Com módulos bem definidos, é possível alterar ou substituir um componente sem impactar toda a arquitetura.



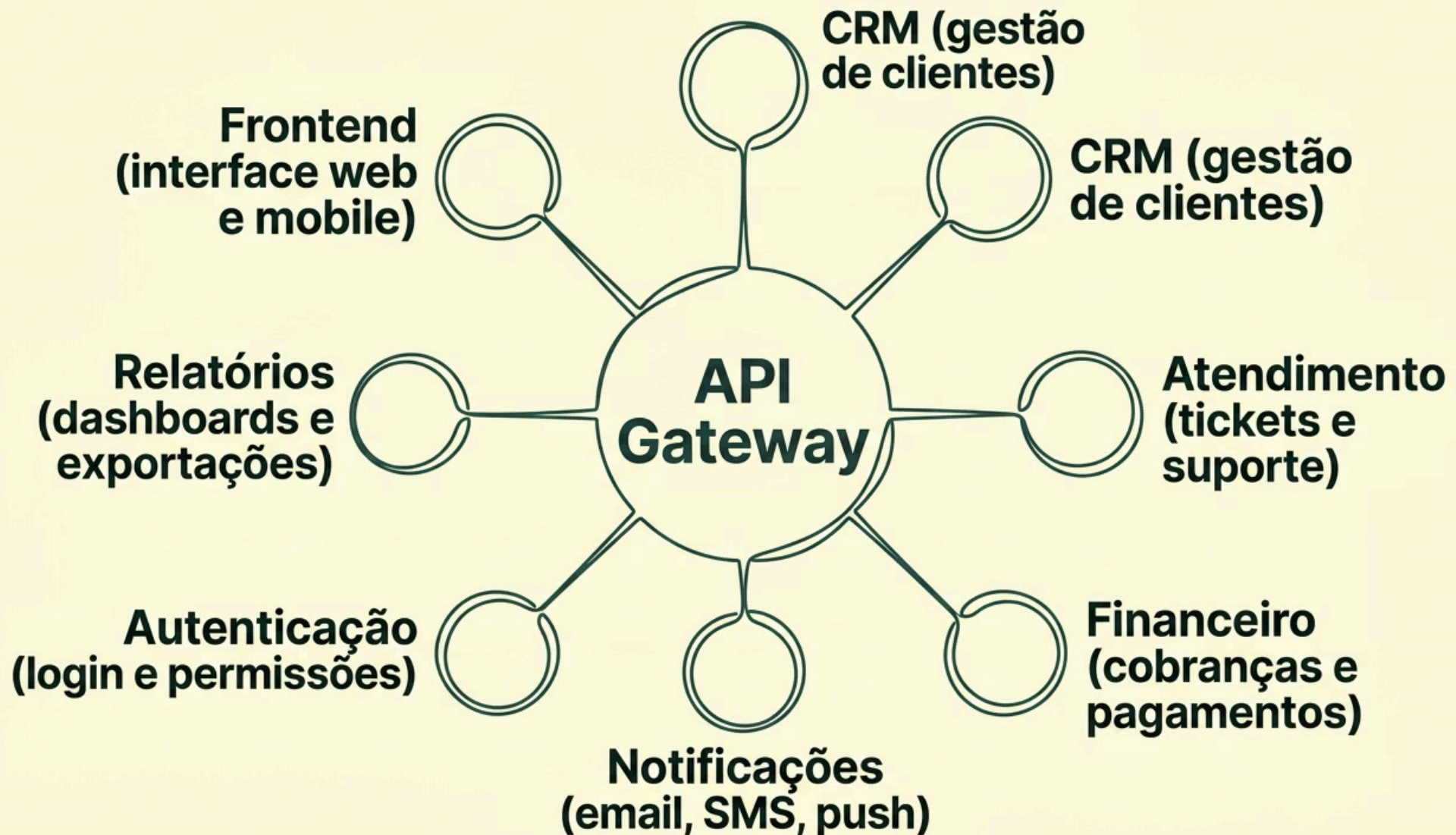
Símbolos Fundamentais do Diagrama

A notação UML define símbolos precisos para representar componentes e seus relacionamentos. Dominar esses símbolos é o primeiro passo para ler e criar diagramas profissionais.

Símbolo	Nome	Descrição
 	Componente	Entidade necessária para executar uma função de estereótipo. Na UML 1.0, bloco retangular com dois retângulos menores saindo pela lateral. Na UML 2.0, bloco retangular com ícone da forma antiga no canto superior direito.
	Nó	Representa objetos de hardware ou software, de nível superior aos componentes.
 	Interface	Mostra entradas ou materiais que o componente recebe ou fornece. Pode ser representada como pirulito, soquete ou "bola-e-soquete".
	Porta	Especifica um ponto de interação separado entre o componente e o ambiente. Representado por um pequeno quadrado.
	Pacote	Agrupar vários elementos do sistema. Representado por pastas de arquivos – pode ser desenhado em torno de vários componentes.
	Nota	Permite que desenvolvedores fixem uma meta-análise ou comentário no diagrama de componentes.
	Dependência	Mostra que uma parte do sistema depende de outra. Representada por linhas tracejadas que vinculam um componente a outro.

Exemplo Prático: Arquitetura de um Sistema SaaS

Em um sistema SaaS moderno, cada funcionalidade de negócio tende a ser encapsulada em seu próprio componente. Veja como um sistema completo se organiza:



A **API Gateway** atua como ponto central de entrada, roteando as requisições para cada componente especializado. Cada módulo pode evoluir, ser testado e implantado de forma independente.

Interfaces: Fornecida vs. Requerida

Interface Fornecida — "Pirulito" ○

O componente **expõe** um conjunto de operações para que outros possam consumir. É um contrato que ele se compromete a cumprir.

Exemplo — API de Usuários fornece:

- `cadastrar(usuario)`
- `consultar(id)`
- `atualizar(usuario)`
- `remover(id)`

Interface Requerida — "Tomada" ⤴

O componente **declara a necessidade** de consumir um serviço externo. Ele não funciona sem que esse serviço esteja disponível.

Exemplo — Serviço de Pedidos requer:

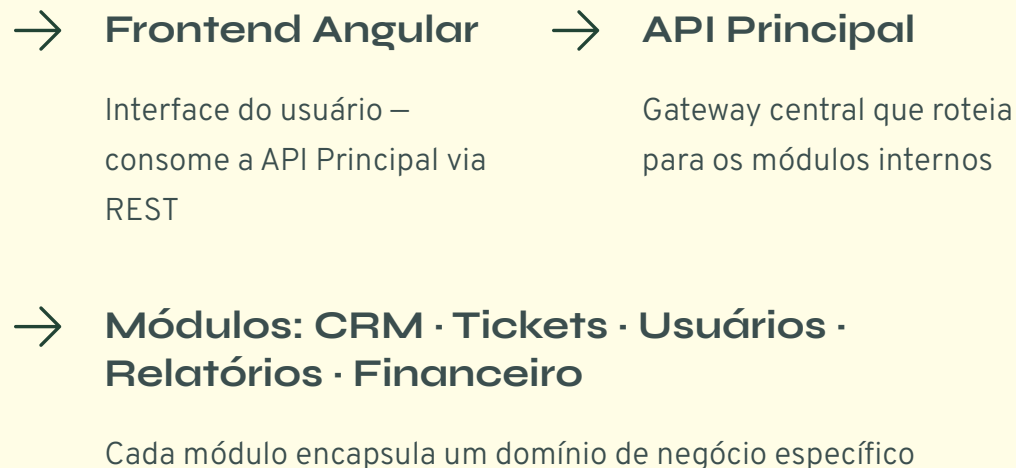
- Interface de Pagamento
- Interface de Notificação
- Interface de Estoque

❏ O par fornecida + requerida forma um **conector de montagem** — a "tomada" se encaixa no "pirulito".

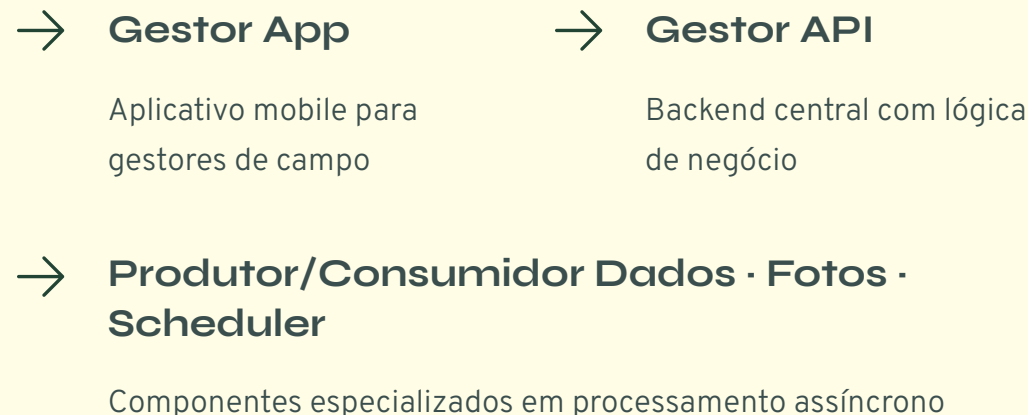
Exemplos Reais de Arquiteturas

Dois exemplos que aproximam a teoria da prática profissional do dia a dia.

Yvy Desk – Sistema de Suporte



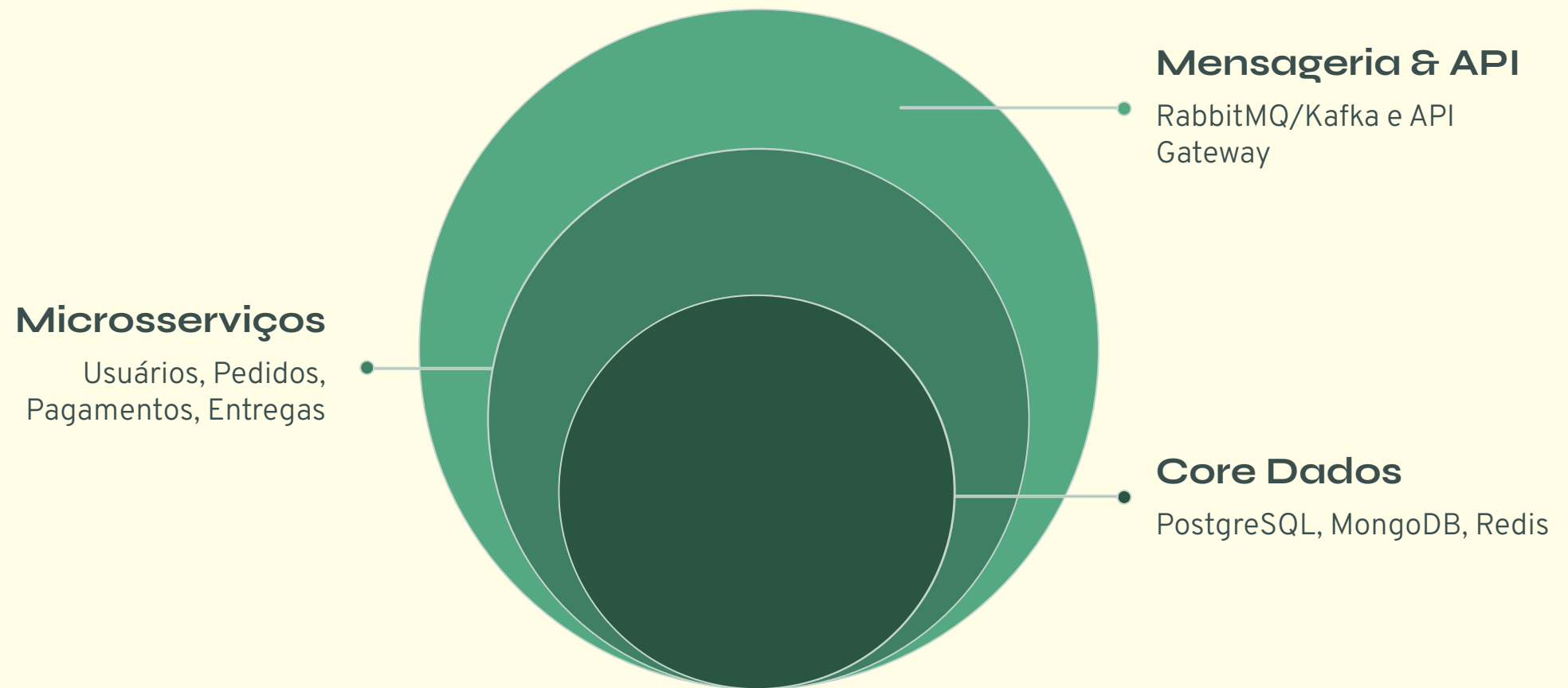
Luz Para Todos – Sistema de Gestão



Perceba que nos dois casos a **API** funciona como ponto central de orquestração, e cada componente tem uma responsabilidade de negócio clara e bem delimitada.

Arquiteturas Modernas com Nuvem e Mensageria

Sistemas contemporâneos frequentemente incorporam brokers de mensagens, bancos de dados gerenciados e serviços em nuvem como componentes de primeira classe no diagrama.



APIs REST

Modeladas como componentes com interfaces fornecidas e requeridas

Mensageria

RabbitMQ, Kafka – componentes de infraestrutura com interfaces de publicação e consumo

Banco de Dados

PostgreSQL, MongoDB – podem e devem aparecer como componentes quando são parte integrante da arquitetura

Erros Comuns ao Criar Diagramas de Componentes

❌ Excesso de Classes no Diagrama

Colocar Cliente, Produto, Pedido como se fossem componentes. Classes são detalhes de implementação – componentes são módulos funcionais.

❌ Misturar Fluxo com Arquitetura

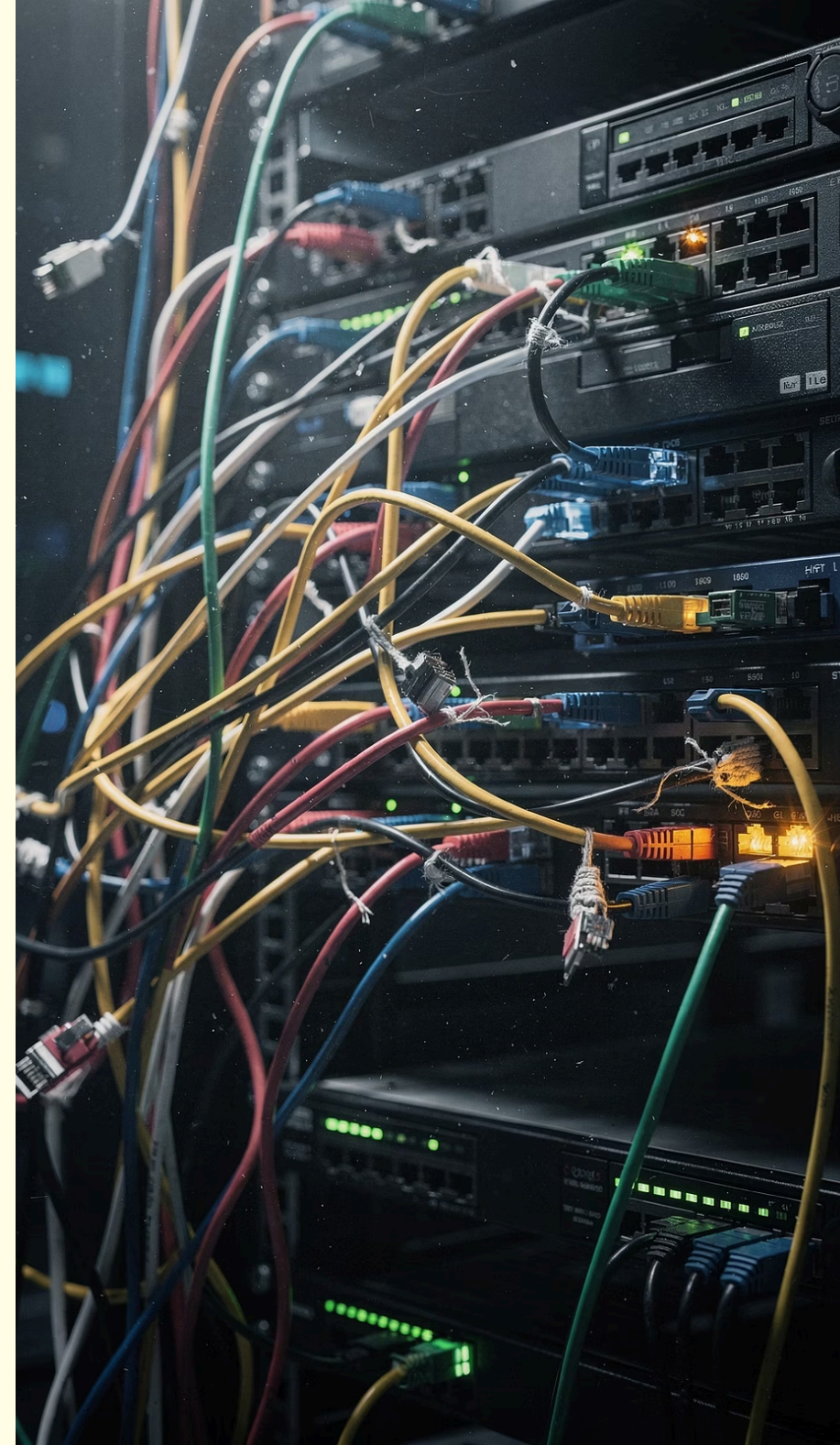
Usar setas numeradas como se fosse um Diagrama de Sequência. Componentes mostram *estrutura*, não *comportamento* ao longo do tempo.

❌ Omitir Dependências

Desenhar componentes soltos, sem mostrar quem depende de quem. O valor do diagrama está exatamente nas conexões.

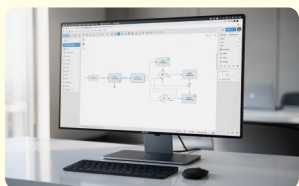
❌ Diagramas Gigantes e Ilegíveis

Tentar colocar todos os 50 módulos em um único diagrama. Prefira diagramas menores por contexto ou domínio de negócio.



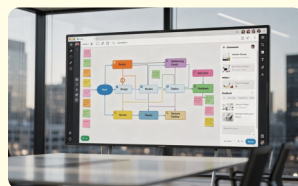
Ferramentas para Criação de Diagramas

Existem excelentes ferramentas – gratuitas e pagas – para criar Diagramas de Componentes UML. Escolha a que melhor se encaixa no seu fluxo de trabalho.



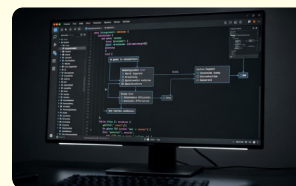
Draw.io

Gratuito, online e com suporte nativo a notação UML. Integra com Google Drive, Confluence e VS Code. Ideal para quem está começando.



Lucidchart

Ferramenta colaborativa em tempo real, muito usada em equipes corporativas. Possui biblioteca rica de formas UML e integração com Jira e Slack.



PlantUML

Diagrama como código – você escreve texto e ele gera o diagrama. Perfeito para versionamento em Git e documentação técnica automatizada.



Structurizr

Especializado em arquitetura de software (modelo C4). Excelente para documentar sistemas complexos com múltiplos níveis de abstração.

Laboratório Final: Modele Seu Próprio Sistema

O aprendizado mais duradouro acontece quando você aplica os conceitos ao seu próprio projeto. Escolha um dos sistemas abaixo ou use o projeto que sua equipe já está desenvolvendo.

1

Sistema de Biblioteca

Componentes: Portal Web, Cadastro de Acervo, Empréstimos, Devoluções, Relatórios, Banco de Dados. Modele as dependências entre cada módulo.

2

Clínica Médica

Componentes: Frontend, Pacientes, Médicos, Consultas, Financeiro, Relatórios, Banco. Desafio extra: adicione um módulo de Notificações por e-mail.

3

Sistema Acadêmico

Componentes: Portal do Aluno, Disciplinas, Matrículas, Notas, Financeiro, Relatórios. Modele também as interfaces fornecidas pelo módulo de Autenticação.

4

Seu Projeto Real

Use o sistema que sua equipe está desenvolvendo agora. Modele a arquitetura atual e proponha melhorias na estrutura de componentes.

- ✔ Dica do instrutor: modelar o próprio projeto gera muito mais aprendizado do que analisar exemplos prontos. O debate em equipe sobre onde colocar as fronteiras dos componentes é, por si só, um exercício valioso de arquitetura.

Como Usar o PlantUML

Ferramenta gratuita para criar diagramas UML com código – sem instalação, direto no navegador.

  Acesse agora: <https://www.plantuml.com/plantuml/uml/>



Acesse o site

Abra o navegador e acesse [plantuml.com/plantuml/uml](https://www.plantuml.com/plantuml/uml/). Nenhuma instalação necessária – funciona 100% online.



Apague o código de exemplo

O editor já vem com um exemplo. Selecione tudo (Ctrl+A) e apague para começar do zero.



Cole ou escreva seu código

Digite os comandos PlantUML: @startuml, component, database, --> e @enduml. O diagrama é gerado automaticamente.



Exporte o diagrama

Clique em "PNG" ou "SVG" para baixar a imagem do diagrama e inserir em documentos, slides ou wikis.



Dica: o PlantUML também pode ser instalado localmente via extensão do VS Code (PlantUML by jebbs) ou integrado ao IntelliJ IDEA para uso offline.

Utilizando o PlantUML

Crie diagramas de componentes com código – rápido, versionável e integrado ao seu fluxo de desenvolvimento.

```
@startuml
title Sistema de Biblioteca - Diagrama de Componentes

skinparam componentStyle rectangle

actor "Usuário" as Usuario

component "Portal Web" as Portal
component "Cadastro de Acervo" as Acervo
component "Empréstimos" as Emprestimos
component "Devoluções" as Devolucoes
component "Relatórios" as Relatorios

database "Banco de Dados" as BD

Usuario --> Portal

Portal --> Acervo : consulta livros
Portal --> Emprestimos : solicita empréstimo
Portal --> Devolucoes : registra devolução
Portal --> Relatorios : acessa relatórios

Emprestimos --> Acervo : verifica disponibilidade
Emprestimos --> BD : registra empréstimo

Devolucoes --> Emprestimos : consulta empréstimo ativo
Devolucoes --> Acervo : atualiza disponibilidade
Devolucoes --> BD : registra devolução

Acervo --> BD : salva/consulta livros
Relatorios --> BD : consulta dados

@enduml
```

component

Declara um componente do sistema. Use aspas para nomes com espaço ou acentos.

database

Símbolo especial para bancos de dados. Outros: actor, node, interface, package.

-->

Define uma dependência/relação entre elementos. Adicione : texto para rotular a seta.

 Teste online em plantuml.com/plantuml – cole o código e veja o diagrama gerado instantaneamente. Também integra com VS Code, IntelliJ e GitHub Actions.